Regarding ModelMapper :

In order to automatically convert Entity <-----> DTO , there are multiple 3rd party libraries.
eg : MapStruct , ModelMapper ...

Steps for using Model Mapper :
1.1 Add model mapper dependency in pom.xml
<!-- https://mvnrepository.com/artifact/org.modelmapper/modelmapper -->
```
  <dependency>
    <groupId>org.modelmapper</groupId>
    <artifactId>modelmapper</artifactId>
    <version>3.0.0</version>
  </dependency>
```

1.2 How to configure ModelMapper object as a spring bean (so that it can be injected as a dependency in other spring beans) ?
Ans : Either use xml <bean> tag OR use
@Bean annotated method , to return ModelMapper instance to SC.

Bean config xml file equivalent to : @Configuration : class level annotation
<bean> Tags --- @Bean : method level annotation(this method should ret bean instance to the caller : SC)

More Details here :

What is the difference between @Bean vs @Component ?
@Component n it’s subtypes are Preferable for component scanning and automatic wiring.
When should you use @Bean?
Sometimes automatic configuration is not an option.
When?
Imagine that you want to wire components from 3rd-party libraries , eg : PasswordEncoder in SpringSecurity or ModelMapper (you don’t have the source code so you can’t annotate its classes with @Component), so automatic configuration is not possible.

The @Bean , method level annotation , which returns an object that spring should register as bean in application context.
The body of the method should create the instance n ret it to the caller (SC) , for it’s auto management as a spring bean.

eg :
```
@Bean //equivalent to <bean id ..../> in xml file
 public ModelMapper modelMapper() {
  ModelMapper modelMapper = new ModelMapper();
  modelMapper.getConfiguration().setMatchingStrategy(MatchingStrategies.STRICT) //only MATCHING prop names between src n dest will be transferred , during the mapping
   .setPropertyCondition(Conditions.isNotNull());// only non null properties will be transferred from src --> dest , during the mapping
 return modelMapper;

 }
```

1.3 Inject ModelMapper as a dependency in other spring beans
(@AutoWired)

1.4 Use ModelMapper’s map method , to map Entity <----> DTO
public D map(T src,Class<D> class)
T : src type of object
D : type of detination.
It will create the instance of destination type class n set MATCHING props from the src object
eg : To map Employee entity ref --> emp resp dto
EmpRespDTO dto=mapper.map(empEntity,EmpRespDTO.class);